BioJEPA-AC: Self-Supervised Cell-State Learning for Perturbation-Response Prediction

Domen Jemec, Bianca Lauro
GPTomics
www.gptomics.com

Abstract

A major challenge for AI in modern biology is using experimental measurements to build a model with a general understanding of cell dynamics and perturbation responses. This report presents a new self-supervised model, the Biological Joint-Embedding Predictive Architecture - Action Conditioned (BioJEPA-AC), that learns a unified latent space to represent cell states and predicts how perturbations move cell representations through that space. BioJEPA-AC v1.0 is a unified 10,000-gene model developed using a corpus of 3.5 million cells from six datasets spanning four cell lines and CRISPRi, CRISPRa, and small-molecule perturbations [7–10]. We evaluate BioJEPA-AC directly in the latent space and, when needed, use simple task-specific heads. Across 1,416 held-out single-perturbation groups, BioJEPA-AC achieves a mean perturbation-level Pearson correlation of 0.332 between predicted and observed expression changes across all genes. Among the 50 genes with the largest measured responses, the mean and median correlations are 0.599 and 0.636. BioJEPA-AC also produces a latent uncertainty signal alongside the predicted state, providing an initial baseline for uncertainty calibration. Together, these results show that BioJEPA-AC v1.0 is an early step toward a virtual-cell model that can generalize across cell states and predict responses to perturbations.

1 Introduction

One of the key challenges in cell and systems biology is being able to accurately predict how a cell or cell system will respond to a change in its environment, such as the introduction of an intervention. Experimental perturbation screens such as Perturb-seq can measure cell responses directly [2], but we cannot feasibly assay every molecule in a cell in every cell type under every possible perturbation. To move faster than relying on measured interventions, we need a model that can generalize to unseen cell types and biological actions and learn the underlying cell physics to make accurate predictions.

We approach this problem with BioJEPA-AC, which draws inspiration from V-JEPA 2-AC's use of actions to predict how a system changes, but substantially adapts the video- and robotic-action-based approach to work with cell states and biological perturbations [17]. Rather than predicting expression directly, the model learns a unified latent space to represent cell states. The action-conditioned predictor takes a cell-state representation as context and a perturbation as an action, then predicts the resulting cell-state representation in that same latent space. To support multiple modalities of perturbations, we train a separate action composer to provide a unified perturbation representation based on the sequence and/or target information. Any perturbation that a biological foundation model can encode from sequence can enter this shared action space. Based on the context cell and perturbation, the model predicts both a latent mean and a corresponding latent variance representation for the perturbed cell state. BioJEPA-AC began with one dataset, a limited gene set, and one perturbation class. In this version, v1.0, we extend the architecture to a unified 10,000-gene model developed using a corpus of ~3.5 million cells from six datasets spanning four cell lines and CRISPRi, CRISPRa, and small-molecule perturbations.

To test whether BioJEPA-AC learns biologically useful cell state representations and can predict how cell states respond to perturbations, we evaluate the model on 1,429 held-out perturbation groups. The perturbed cells in these groups were held out during encoder training, their perturbations were held out during action composer training, and the complete groups, including their randomly paired batch-specific control cells, were held out during predictor and decoder training. This design ensures that the evaluations measure generalization to new perturbations within familiar cellular settings. We evaluate the latent representations directly and, when interpretable outputs are needed, use simple task-specific heads. These evaluations examine how well the model captures perturbation-induced changes in expression across the full gene set and among the strongest measured responses, as well as whether its latent variance provides a meaningful uncertainty signal. We also assess its

current ability to model combination perturbations. Overall, the results reported here represent a focused subset of the broader evaluation suite we developed to assess BioJEPA-AC as a world model for cells.

2 RESULTS

2.1 ONE SHARED MODEL ACROSS CELL LINES AND PERTURBATION TYPES

BioJEPA-AC v1.0 uses one shared 10,000-gene model across six datasets, four cell lines, and three perturbation types. Its primary benefit is not only the joint-embedding space itself, but also its ability to take actions, in our case perturbations, and move cell representations through that space. To evaluate whether the learned latent space and ability to move representations across it are useful, we evaluate both the latent space and, when needed, its performance through task-specific decoder heads.

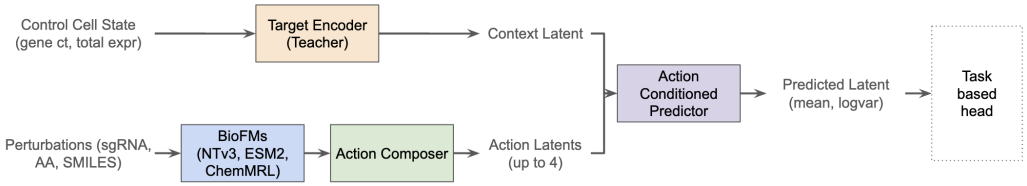

Fig. 1: *The BioJEPA-AC inference forward pass treats a control cell as the context and generates the perturbed cell's latent representations, which are used by the evaluations.*

To run the model evaluations, we take a random control cell from a batch, encode it to generate a cell state, add up to four perturbation representations, and run an inference pass through BioJEPA-AC. This generates a predicted perturbed cell state representation that we then pass to the corresponding evaluation or task-specific head. Across the six datasets, the test set contains 339,009 sample pairs representing 1,429 perturbation groups held out from predictor training. A perturbation group is the collection of test samples that share the same perturbation identity, target, modality, mode of action, and cell line. We report both our overall evaluation performance and a per-dataset breakdown.

2.2 EXPRESSION PREDICTION

This evaluation measures how well our model predicts post-perturbation gene expression using a single-layer decoder head. Accurate expression prediction provides an interpretable view into a perturbation's effects on viability and gene networks. Since most of our 10,000 genes do not change substantially under any one perturbation, metrics on absolute expression can reward the model for preserving the control state rather than predicting the perturbation response. We therefore focus on predicted changes in expression, or *expression deltas*. We also analyze performance across the full gene set and among the genes with the largest measured responses. Since each dataset does not actually contain all 10,000 genes, we can only evaluate the expression delta prediction against the gene expression counts available in each dataset.

For evaluations focused on differentially expressed genes (DEGs), we select the genes with the largest absolute *real delta* at the corresponding evaluation level. Since this selection is based on the measured response, these metrics evaluate how well BioJEPA-AC predicts the dominant response program rather than whether it can identify DEGs in advance.

2.2.1 Sample-Level Evaluation

Sample-level expression evaluations calculate performance for each sample in the test set. We create each sample by pairing a case cell with a batch-specific control cell and its perturbation information.

2.2.1.1 Top 20 DEGs Pearson Correlation Coefficient

For each sample, we calculate Pearson's r between the *predicted delta* and *real delta* on the top 20 DEGs. We use only the top 20 genes at the sample level since individual samples have noisier delta estimates than perturbation-level means. Pearson correlation ignores magnitude and focuses on the relative expression pattern, so it determines whether our model captures that pattern among the genes with the largest measured changes.

Dataset	BioJEPA-AC v1.0
Overall	*0.8654*
Adamson	0.8601
Replogle K562 essential	0.8824
Replogle K562 genome-wide	0.9062
Norman	0.7987
Replogle RPE1 essential	0.8561
SciPlex	0.7615

2.2.2 Perturbation-Level Evaluation

Perturbation-level evaluations aggregate sample-level performance across the 1,416 single-perturbation groups, with a unique perturbation defined as a distinct combination of sequence, target, modality, mode, and cell line. Observed and predicted expression changes are calculated at the sample level first, then the mean is calculated across all samples for each perturbation. We evaluate the 13 combination groups separately.

2.2.2.1 All-Gene Expression Delta Pearson Correlation Coefficient

Since expression for most genes remains unchanged, we expect that evaluating across all gene deltas will have a mean close to zero and a low variance. This means that a few changed genes have an outsized influence on this metric, increasing its difficulty.

Dataset	BioJEPA-AC v1.0
Overall	*0.3324*
Adamson	0.2794
Replogle K562 essential	0.2756
Replogle K562 genome-wide	0.3429
Norman	0.3298
Replogle RPE1 essential	0.3970
SciPlex	0.2349

On the same Replogle K562 essential benchmark and GEARS split, the AttentionPert benchmark reports an all-gene Pearson delta of 0.327 ± 0.015 for GEARS, compared with 0.276 for BioJEPA-AC [16]. While BioJEPA-AC does not exceed GEARS on this matched benchmark, v1.0 uses one shared model across multiple datasets, cell lines, and perturbation types.

2.2.2.2 Top 50 DEGs Pearson Correlation Coefficient

We also calculate Pearson's correlation per perturbation on the top 50 DEGs. This focuses the evaluation on the dominant response program and measures whether the predicted and real deltas follow the same relative pattern across the genes with the largest measured changes.

Dataset	BioJEPA-AC v1.0
Overall	*0.5993 (median 0.6365)*
Adamson	0.4139
Replogle K562 essential	0.4793
Replogle K562 genome-wide	0.6458
Norman	0.3368
Replogle RPE1 essential	0.5986
SciPlex	0.4116

The higher top-50 correlation of 0.5993, compared with 0.3324 across all genes, shows that BioJEPA-AC more closely predicts the relative pattern among the largest measured expression changes than across the complete gene panel.

2.3 Gene-Level Analysis

This evaluation assesses whether our model correctly predicts the direction of expression change among the genes most affected by a perturbation. Unlike Pearson's correlation, which measures how the predicted and observed expression profiles vary together, directional accuracy measures how well each gene is classified as upregulated, downregulated, or unchanged.

2.3.1 Directional Accuracy for Top 50 DEGs

Using the same top 50 DEGs as the perturbation-level Pearson evaluation, we classify both the *real delta* and *predicted delta* for each gene as upregulated, downregulated, or unchanged using a direction threshold of

$\tau = 0.25$. Directional accuracy measures how often the predicted and observed classifications agree. This complements the expression-correlation results by testing whether the model places the strongest measured responses on the correct side of the direction threshold.

Dataset	BioJEPA-AC v1.0
Overall	*0.7185*
Adamson	0.4545
Replogle K562 essential	0.4517
Replogle K562 genome-wide	0.7483
Norman	0.5320
Replogle RPE1 essential	0.3637
SciPlex	0.7026

2.4 Uncertainty Estimation and Calibration

This evaluation assesses how meaningful our model's uncertainty estimates are. BioJEPA-AC outputs both a mean and log-variance latent representation for the perturbed cell state. A useful uncertainty signal should assign greater uncertainty to predictions with larger expression errors. Given that there was no dedicated uncertainty training, these results are reported as a v1.0 calibration baseline.

2.4.1 Expected Calibration Error (ECE)

For each sample, we average the predicted latent log-variance to produce a single uncertainty score and compare it with the mean squared error (MSE) between the *predicted delta* and *real delta* across the genes measured in that dataset. ECE quantifies the gap between normalized uncertainty and normalized expression error across 10 uncertainty bins. Lower values indicate closer calibration, with ECE = 0 representing perfect agreement.

Dataset	BioJEPA-AC v1.0
Overall	*0.3232*
Adamson	0.0593
Replogle K562 essential	0.1199
Replogle K562 genome-wide	0.4803
Norman	0.1114
Replogle RPE1 essential	0.2244
SciPlex	0.1932

BioJEPA-AC achieves an overall sample-level ECE of 0.3232. However, the 25% of samples assigned the lowest uncertainty have an MSE of 0.2165, compared with 0.1949 across all samples. The current latent variance therefore does not yet reliably identify more accurate predictions.

2.5 Combination Perturbation Impact

While BioJEPA-AC supports combinations of up to four perturbations, combination generalization is not established in v1.0. The available evaluation contains only 13 combinations from a single dataset (Norman). We need a larger combination holdout before we can assess whether the model has meaningfully learned non-additive genetic interactions.

3 Discussion

The goal with BioJEPA-AC is that a model can learn a shared representation of cell state and that perturbations can act as the actions that move cells through that latent space. In the model, the encoder represents the starting cell state, the perturbation composer places differing perturbations into a shared action space, and the predictor uses the cell state and action to generate the predicted perturbed-cell representation. We train these components in separate stages, but they work together to learn the relationship between the starting cell state, the perturbation, and the resulting cell state. Our hypothesis is that the latent space learned through this training encodes aspects of the underlying cell physics that govern cell dynamics and how they respond to environmental changes such as perturbations.

The evaluations of BioJEPA-AC v1.0 show that the predicted cell state contains biologically useful structure. The model captures the pattern, direction, and severity of perturbation responses. We also evaluate broader properties of the representation, including its ability to identify responsive genes, organize relationships between perturbations, track dose-dependent effects, compose actions, and estimate uncertainty. Several of these evaluations produce above-random performance using the same base BioJEPA-AC model without task-specific tuning of the model itself. The main result of v1.0 is that one shared model can represent and apply perturbations

across the biological settings included in the training corpus. The current holdout establishes generalization to new perturbations within learned cellular contexts. Extending the same framework into new cellular contexts is the next generalization step.

3.1 Future Directions

Future work will improve the capabilities present in v1.0 while expanding the biological coverage of the model. This includes custom uncertainty training with held-out calibration, as well as training matched models at several sizes to derive a scaling law for the architecture. We will expand predictor training with more combination perturbations and broaden the shared perturbation space with additional perturbations, including protein-based ones. Because the training data is heavily skewed toward K562, we will add more examples from underrepresented cell lines and introduce new cell types. For evaluation, we will test whether the model can identify DEGs prospectively and use full cell-type holdouts to determine whether these changes support generalization to new cellular contexts.

The structure of BioJEPA-AC allows us to use different sources of training data for each component, though each expansion will require retraining the full model. The encoder can learn cell-state representations from diverse scRNA-seq datasets without requiring paired perturbations and can be expanded to incorporate non-RNA measurements of cell state, such as cellular images or protein abundance. The perturbation composer can learn from larger catalogs of known perturbations without requiring a measured cellular response. Paired Perturb-seq data can then be reserved for predictor training, where it is needed to learn how perturbations move cells through the latent space. This would broaden the biological data used to build both representations while using paired perturbation measurements specifically to learn the relationship between them.

4 Methods

4.1 Evaluation

4.1.1 Linear Expression Decoder

BioJEPA-AC outputs an embedding in the cell state latent space rather than directly interpretable expression values. To evaluate biological utility, we freeze BioJEPA-AC and train an intentionally simple linear expression decoder so that the evaluation primarily reflects the structure learned by the base model.

Since our cell state latent representation is $[\text{n\_genes}, \text{embed\_dim}]$, the decoder projects each gene's embedding to a scalar expression value through its single shared linear layer. We freeze BioJEPA-AC and train the decoder for five epochs using mean squared error on measured-gene expression deltas, a batch size of 32, a learning rate of 10^{-3}, and OneCycleLR scheduling [28]. The predictor outputs both mean and log-variance latent representations, but we feed only the mean into the decoder, reserving the log-variance for uncertainty analysis.

4.1.2 Evaluation Data Splits: Holdout

To create our held-out datasets, we target an 85%/5%/10% training/validation/test split, with actual percentages varying slightly by dataset due to cross-dataset perturbation overlap. We use the GEARS Python package [11] to identify the held-out training/validation/test split for Replogle K562 essential [8]. The GEARS split uses a 67.5%/7.5%/25.0% training/validation/test split, which we adopt as our starting point. The split identifies unique perturbations to hold out. Starting from this, we build a holdout perturbation list that ensures we hit 15% held-out perturbations per dataset and that those perturbations are held out across all datasets. This ensures we have no perturbation leakage and allows a direct head-to-head comparison on the Replogle K562 essential dataset using the GEARS-defined held-out set.

For the remaining datasets, where we selected held-out perturbations at random, we report BioJEPA-AC results by dataset but do not compare them directly with published numbers. Differences in data splits, preprocessing, aggregation, gene selection, and metric definitions prevent an interpretable comparison. Replogle K562 essential on the GEARS-defined split is the only dataset where we make a direct numeric comparison.

4.1.3 Expression Prediction Metrics

Our *real delta* is the observed change in expression: $\delta_g = x_g^{\text{case}} - x_g^{\text{ctrl}}$. To get our *predicted delta*, we pass both the predicted and control latent representations through the same linear expression decoder and compute the difference: $\hat{\delta}_g = \hat{x}_g^{\text{case}} - \hat{x}_g^{\text{ctrl}}$. We use the predicted control expression to evaluate expression change in terms of BioJEPA-AC's learned latent space, ensuring that if the model learns a different baseline for the control but knows the distance to move the case cell, it can still score well.

4.1.3.1 Sample-Level Top 20 Pearson Correlation

For each sample, we calculate Pearson's r between the *predicted delta* and *real delta* on the top 20 DEGs, selecting the genes with the largest absolute *real delta*. We calculate:

$$\text{Pearson } r_{\text{sample}} = \frac{1}{N}\sum_{i=1}^{N}\frac{\sum_{k=1}^{K}(\hat{\delta}_{i,k}-\bar{\hat{\delta}}_i)(\delta_{i,k}-\bar{\delta}_i)}{\sqrt{\sum_{k=1}^{K}(\hat{\delta}_{i,k}-\bar{\hat{\delta}}_i)^2}\cdot\sqrt{\sum_{k=1}^{K}(\delta_{i,k}-\bar{\delta}_i)^2}}$$

Here, N is the number of sample pairs, and $K = 20$ genes are selected independently for each sample based on the largest $|\delta_{i,k}|$. The bars are the means across the 20 selected genes for that sample.

4.1.3.2 Perturbation-Level Expression Correlations

We define a unique perturbation as a distinct combination of sequence, target, modality, mode, and cell line. We first compute *real delta* and *predicted delta* at the sample level, then take the mean across all samples for each perturbation. This results in per-perturbation *real delta* and *predicted delta* values. We then calculate Pearson's correlation per perturbation on expression deltas for all genes. For the DEG-based metric, we use the same calculation on the 50 genes with the largest observed absolute expression delta. For a detailed breakdown of both calculations, see our expression-prediction explainer notebook.

4.1.4 Gene-Level Directional Accuracy

We classify each gene's perturbation-level mean expression delta into one of three categories:

$$d(\delta_k) = \begin{cases} +1 & \text{if } \delta_k \geq \tau \\ -1 & \text{if } \delta_k \leq -\tau \\ 0 & \text{otherwise} \end{cases}$$

Here, $\tau = 0.25$ is the direction threshold, representing a difference of 0.25 in normalized log-expression between the control and perturbed states. We apply this classification to both our real and predicted perturbation-level mean expression deltas, then select the top 50 genes per perturbation based on the largest absolute real expression delta. We calculate:

$$\text{Direction Accuracy}_{topK} = \frac{1}{P\cdot K}\sum_{p=1}^{P}\sum_{k\in\mathcal{T}_p}\mathbb{1}\left[d(\hat{\delta}_{p,k}) = d(\delta_{p,k})\right]$$

Here, P is the number of perturbation groups and $\mathcal{T}_p$ is the set of $K = 50$ genes with the largest $|\delta_{p,k}|$ for perturbation p. For a detailed breakdown, see our gene-analysis explainer notebook.

4.1.5 Uncertainty Metrics

The predictor returns a log-variance for every latent position and channel. For each sample, we first average across the $D = 256$ latent channels at each position, then average across the K positions corresponding to genes measured in that dataset to produce a single uncertainty score. We calculate expression error as the MSE between the *predicted delta* and *real delta* across the same genes:

$$\ell_{s,k} = \frac{1}{D}\sum_{d=1}^{D}\text{logvar}_{s,k,d}$$

$$u_s = \frac{1}{K}\sum_{k=1}^{K}\ell_{s,k}$$

$$e_s = \frac{1}{K}\sum_{k=1}^{K}(\hat{\delta}_{s,k}-\delta_{s,k})^2$$

Here, u_s and e_s are the scalar uncertainty and expression MSE for sample s. To calculate ECE, we min-max normalize both uncertainty and MSE to $[0, 1]$, then partition samples into 10 equal-width bins by normalized uncertainty:

$$\text{ECE} = \sum_{b=1}^{10} \frac{|\mathcal{B}_b|}{N} |\bar{u}_b - \bar{e}_b|$$

Here, $\mathcal{B}_b$ is the set of samples in bin b, N is the total number of samples, and $\bar{u}_b$, $\bar{e}_b$ are the mean normalized uncertainty and mean normalized error within bin b.

We also sort samples by u_s and compare the mean expression error among the 25% assigned the lowest uncertainty with the mean across all samples:

$$\text{MSE}_{\text{lowest 25\%}} = \frac{1}{|\mathcal{S}_{25}|} \sum_{s \in \mathcal{S}_{25}} e_s, \qquad \text{MSE}_{\text{all}} = \frac{1}{N} \sum_{s=1}^{N} e_s$$

Here, $\mathcal{S}_{25}$ contains the 25% of samples with the lowest uncertainty scores. For a detailed breakdown, see our uncertainty-calibration explainer notebook.

4.2 Model Training

BioJEPA-AC is trained in three stages with a separate objective for each stage. The encoder uses Warmup-Stable-Decay scheduling, which linearly warms up the learning rate, holds it constant, and then linearly decays it toward zero. Since the current encoder run stops after three epochs of the ten-epoch schedule, it completes during the stable phase and does not reach the decay phase. The action composer and predictor use OneCycleLR, which linearly warms up the learning rate from near-zero to the maximum over the first 5% of training, then cosine-decays it back down to near-zero. We also reshuffle training data shards each epoch to prevent learning from sequential ordering. For a detailed breakdown of training loss, see our explainer notebook.

4.2.1 Action Composer Training

The first component we train is the action composer. Using contrastive learning, the composer learns a unified embedding space for perturbation modalities and modes of action. The composer training split contains 10,791 sequence-target pairs. While BioJEPA-AC can handle sequence-only or target-only perturbations for inference, contrastive learning requires both, so the composer training can use only sequence-target pairs. To ensure that chemical perturbations are not fully diluted out during training, we duplicate the chemical pairs so that they are approximately 10% of the training pool. This results in a total of 11,776 chemical and genetic training pairs per epoch. We train for 10,000 epochs, reshuffling the samples each epoch to vary the negative pairs within each batch. Since we are trying to learn how perturbations share an embedding space, we present each perturbation independently during training, even when datasets contain multi-perturbation samples. During predictor training, we also update the composer weights at 1% of the predictor's learning rate to fine-tune the composer based on prediction accuracy.

For the action composer training, we use InfoNCE [13] loss, which is the cross-entropy over a temperature-scaled cosine similarity matrix between the sequence and target path representations. We calculate training loss as:

$$\mathcal{L}_{align} = -\frac{1}{B} \sum_{i=1}^{B} \log \frac{\exp(\hat{z}_{seq,i} \cdot \hat{z}_{target,i}/\tau)}{\sum_{j=1}^{B} \exp(\hat{z}_{seq,i} \cdot \hat{z}_{target,j}/\tau)}$$

Here, $\hat{z}_{seq}$ and $\hat{z}_{target}$ are L2-normalized representations from the sequence and target paths, and $\tau = 7.559 \times 10^{-4}$ is the temperature. The low temperature sharpens the similarity distribution, forcing the model to strongly prefer the correct pair over all other perturbations in the batch. Optimal training parameters were found using Bayesian optimization with Optuna [29].

4.2.2 Cell State Encoder Training

The second component we train, which can run in parallel with the action composer training, is the cell state encoder. During this stage, the student serves as the context encoder and the teacher provides the target representations. After training, we freeze both encoders and use the teacher as the cell state encoder for predictor training, decoder training, and inference. The encoder training split contains 2,999,193 cells. During training, we balance the dataset shards so that each dataset has at least 20% as many train shards as the largest dataset. This repeats Adamson six times, Replogle K562 essential and RPE1 twice, and Norman four times, while Replogle K562 genome-wide and SciPlex are not repeated. This results in a total of 4,065,280 samples per epoch and we train for three epochs.

We use masked training, masking 76.6% of the gene positions (identified via Bayesian optimization with Optuna [29]). The target encoder starts as a copy of the context encoder. During training, the student receives a masked

cell state while the teacher sees the complete unmasked cell state and provides the target latents. We update the teacher's weights using the exponential moving average of the student's weights rather than gradients [27], providing a slowly changing target representation that creates a smoother training signal.

We use a combination of L1 loss and variance-invariance-covariance regularization (VICReg) [12] loss. We calculate training loss as:

$$\mathcal{L}_{encoder} = \lambda_{sim} \cdot \underbrace{\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|}_{\text{L1 reconstruction}} + \lambda_{std} \cdot \underbrace{\frac{1}{d}\sum_{j=1}^{d}[\text{ReLU}(1-\sigma_j(y)) + \text{ReLU}(1-\sigma_j(\hat{y}))]}_{\text{std loss}} + \lambda_{cov} \cdot \underbrace{\frac{1}{d}\sum_{i\neq j}\left[C_{ij}(y)^2 + C_{ij}(\hat{y})^2\right]}_{\text{cov loss}}$$

The masked predictor uses two transformer layers. The L1 term, weighted by $\lambda_{sim} = 50.18$, measures how accurately it reconstructs the teacher's latents at masked gene positions measured in that dataset. Unmeasured and unmasked gene positions do not contribute to the reconstruction loss. We calculate the VICReg terms across all latent positions, using $\lambda_{std} = 25.44$ and $\lambda_{cov} = 0.5158$. The std term penalizes any feature dimension whose variance drops below 1, and the cov term penalizes correlations between feature dimensions, together preventing representation collapse.

4.2.3 Action-Conditioned Predictor Training

After training the encoder and composer, we train the action-conditioned predictor. The predictor learns to traverse the cell state latent space learned by the encoder. The predictor training split contains 2,856,660 examples, each consisting of the perturbation, the perturbed cell, and a random control cell from the same batch. We balance the dataset shards so that each dataset has at least 20% as many train shards as the largest dataset. This repeats Adamson six times, Replogle K562 essential and RPE1 twice, and Norman four times, while Replogle K562 genome-wide and SciPlex are not repeated. This results in a total of 3,362,112 samples per epoch and we train for five epochs.

We freeze both encoders (student and teacher) and train the predictor and composer together, with the composer receiving 1% of the predictor's learning rate. During this stage, we apply 15% dropout independently to the composer's sequence and target representations before adding them together. This encourages the composer to remain useful when either source of perturbation information is incomplete. We apply gradient clipping (norm=5.0) to the predictor and action composer parameters for stability. We use two annealing schedules. Beta-NLL annealing ramps β from 0 to 0.95 over the first 40% of training. As β increases, the model can no longer rely on increasing its predicted uncertainty to lower the loss and must instead improve the accuracy of its mean predictions. Mask-ratio annealing decreases the percentage of masked gene positions from 76.6% to 0% over the final 40%. This gradually gives the predictor more control-cell context and allows it to finish training under conditions closer to inference.

We use a combination of beta-weighted Gaussian NLL [14] loss and VICReg [12] loss. We calculate training loss as:

$$\mathcal{L}_{predictor} = \lambda_{sim} \cdot \underbrace{\frac{1}{|\mathcal{M}|}\sum_{i\in\mathcal{M}}\text{stopgrad}(v_i^{\beta}) \cdot \frac{1}{2}\left(\log v_i + \frac{(y_i-\mu_i)^2}{v_i}\right)}_{\beta\text{-weighted Gaussian NLL}} + \lambda_{std} \cdot \underbrace{\frac{1}{d}\sum_{j=1}^{d}[\text{ReLU}(1-\sigma_j(y)) + \text{ReLU}(1-\sigma_j(\mu))]}_{\text{std loss}} + \lambda_{cov} \cdot \underbrace{\frac{1}{d}\sum_{i\neq j}\left[C_{ij}(y)^2 + C_{ij}(\mu)^2\right]}_{\text{cov loss}}$$

Here, $\mathcal{M}$ is the set of latent elements belonging to measured genes and $v_i = \exp(\text{logvar}_i)$ is the predicted variance for element i. The Gaussian NLL term is calculated over latent elements at measured gene positions. It is weighted by $\lambda_{sim} = 50.18$, which penalizes inaccurate latent mean predictions and variance estimates that do not match the latent prediction error at measured gene positions. Masking changes how much of the control cell is available as context, but does not change which positions contribute to the loss. We detach the v_i^{β} weighting from the gradient calculation, reducing how much high predicted variance suppresses the error signal without allowing the weighting term itself to drive the predicted variance.

We calculate the VICReg terms across all latent positions, using $\lambda_{std} = 25.44$ and $\lambda_{cov} = 0.5158$ (the same values used for the encoder).

4.3 Data Preparation

4.3.1 Data Sources

We train BioJEPA-AC on six publicly available single-cell perturbation datasets spanning three perturbation types (CRISPRi, CRISPRa, chemical), both single and multi-perturbation applications, and four cell lines (K562, RPE1, A549, MCF7). Together, these provide ~3.5 million cells for training and evaluation. Our primary

dataset is the Replogle K562 essential screen [8], for which we use the GEARS-defined [11] held-out splits as our evaluation starting point.

Dataset	Mode & Cell Line	Total Cells	Batches	Genes in 10K	Eval Sample Pairs	Eval Perturb Groups*
Adamson [7]	CRISPRi K562	65,337	1	9,749	4,288	11
Replogle K562 essential [8]	CRISPRi K562	310,385	48	8,563	46,506	286
Replogle K562 genome-wide [8]	CRISPRi K562	1,989,578	267	8,248	186,535	1,053
Norman [9]	CRISPRa K562	111,445	8	9,865	8,001	23
Replogle RPE1 essential [8]	CRISPRi RPE1	247,914	56	8,162	22,838	287
SciPlex [10]	Chemical A549, K562, MCF7	799,317	52	9,974	70,841	54

*Perturbation groups represented in multiple datasets are counted in each dataset row but only once in the overall total, meaning that the 1,714 per-dataset groups represent 1,429 unique groups: 1,416 single and 13 combination perturbations.

4.3.2 Gene Universe

We define a unified gene space of 10,000 genes across all datasets. Since not every dataset measures the same genes, we use a priority-based and expression-level selection strategy. We start with all 8,563 genes from the Replogle K562 essential dataset. We then filter genes in the remaining datasets to those expressed in at least 10 cells, pool the candidates, and fill the remaining 1,437 positions with the most broadly expressed genes ranked by total cell count across datasets.

Since the datasets measure different gene panels, we generate per-dataset gene masks that record which genes each dataset actually measured. Genes outside a dataset's panel are zero-filled when we store the expression vector, but we do not treat them as observed genes with zero expression. Before passing a cell through the encoder, we replace these positions with a learned unknown token. We also exclude unmeasured positions from the encoder reconstruction loss and predictor Gaussian NLL.

4.3.3 Expression Normalization

We normalize raw count data using counts-per-ten-thousand (CP10K) followed by log1p:

$$\tilde{x}_g = \log\left(1 + \frac{x_g}{\sum_{g'=1}^{G} x_{g'}} \cdot 10{,}000\right)$$

Here, x_g is the raw count for gene g and G is the total number of genes. This corrects for sequencing depth differences while the log transform compresses the heavy-tailed count distribution, so the loss treats high and low expression genes more equally.

We also preserve the cell's total expression as a separate input to the encoder. We normalize the raw total relative to the mean total for its experimental batch and apply log1p:

$$t = \log\left(1 + \frac{\text{raw cell total}}{\text{batch mean total}}\right)$$

When a batch-specific mean is unavailable, we use the dataset-level mean. This retains information about the cell's overall expression level that is removed by CP10K normalization. For predictor shard generation, we exclude perturbed-cell examples whose perturbation lacks both a sequence and target embedding. Encoder shard generation does not require perturbation embeddings. For both stages, we filter cells with fewer than 50 expressed genes.

4.3.4 Control Matching

For each dataset, we build a control bank by identifying untreated cells (labeled as control, non-targeting, or similar) and grouping them by experimental batch. For each perturbed cell, we pair it with random control cells from its batch, or, if the batch is unavailable, a random control from the full pool.

4.3.5 Train, Validation, and Test Splits

The holdout strategy differs by training stage. For encoder training, perturbed cells follow the perturbation-level split, while control cells are randomly assigned approximately 85% to training, 5% to validation, and 10% to

test. For the action composer, perturbations are held out based on the same target list. For predictor training, the perturbed cell, matched control cell, and perturbation are held out together.

We define held-out perturbations starting from the GEARS-defined [11] split for Replogle K562 essential (734 train, 82 validation, 272 test), giving us a direct comparison to published benchmarks. For full details on the cross-dataset split strategy, see Evaluation Data Splits.

4.3.6 Perturbation Embeddings

We pass perturbation sequences and targets through publicly available foundation models during data preparation, producing the precomputed embeddings our action composer takes as input.

4.3.6.1 Sequence Embeddings

For genetic perturbations, the sequence is the 20-nucleotide protospacer (guide RNA target) used in the CRISPR experiment. We embed each protospacer using the Nucleotide Transformer v3 (650M parameters) [5], producing 1,536-dimensional vectors. For CRISPRi experiments with dual-guide designs (sgID_A and sgID_B), we embed each guide separately and average. For Norman CRISPRa dual-gene perturbations, we similarly embed each gene's protospacer and average.

For chemical perturbations, the sequence is the drug's SMILES string. We look up SMILES for each of the 188 SciPlex compounds via PubChem [18], then embed them using ChemMRL [15], producing 1,024-dimensional vectors. We zero-pad these vectors to 1,536 dimensions for shared storage and batching with DNA embeddings. The action composer uses the first 1,024 dimensions when projecting the chemical embeddings.

In total, we generate 11,643 DNA embeddings and 188 chemical embeddings.

4.3.6.2 Target Embeddings

The perturbation target is the protein product of the perturbed gene for genetic perturbations or the drug's known protein target for chemical perturbations. If a perturbation has multiple targets, we currently take the first. We map gene identifiers to UniProt accessions via MyGene.info [19], retrieve protein sequences from the UniProt human proteome [20], and embed them using ESM-2 (8M parameters) [6], producing 320-dimensional vectors. For genes missing from UniProt, we fall back to Entrez [21] and Ensembl [22] protein lookups. We successfully map 9,975 of 9,985 target genes to protein sequences, with the 10 unmapped targets being noncoding RNAs.

4.3.6.3 Alignment Pairs

For action composer training, we need paired sequence and target examples where both embeddings exist. The alignment shards contain 10,808 CRISPRi pairs, 232 CRISPRa pairs, and 188 chemical inhibitor pairs, for a total of 11,228. We split these into 10,791 training, 108 validation, and 329 test pairs.

4.3.6.4 Missing Embedding Handling

Some perturbations lack one or both embeddings. For predictor training, we require either a sequence or a target, and the action composer handles the missing path via pass-through. We exclude samples linked to the four perturbations missing both embeddings entirely.

4.3.7 Shard Generation

We convert the processed data into compressed NumPy archives (shards) for streaming during training. We generate separate shards for encoder, action composer, and predictor training since each stage requires different data layouts. Encoder and predictor shards contain up to 2,560 cells.

5 Data and Model/Code Availability

5.1 Data Availability

BioJEPA-AC v1.0 uses six publicly available single-cell perturbation datasets consisting of single-cell RNA-seq data. Each source dataset is available through its corresponding Adamson, Replogle, Norman, or SciPlex publication [7–10]. We describe the gene-universe construction, normalization, control matching, perturbation holdouts, and shard generation used for this report in Data Preparation.

5.2 Model and Code Availability

The BioJEPA model, training, and evaluation code is available in the BioJEPA repository. The repository also contains the linked explainer notebooks for the cell state encoder, action composer, action-conditioned predictor,

training objectives, and evaluation calculations. Checkpoints can be provided on request.

6 ACKNOWLEDGEMENTS

We thank Jason Chin for insightful discussions that informed our thinking about the design, evaluation, and potential applications of BioJEPA-AC.

A BioJEPA-AC Model Specification

A.1 Architecture Overview

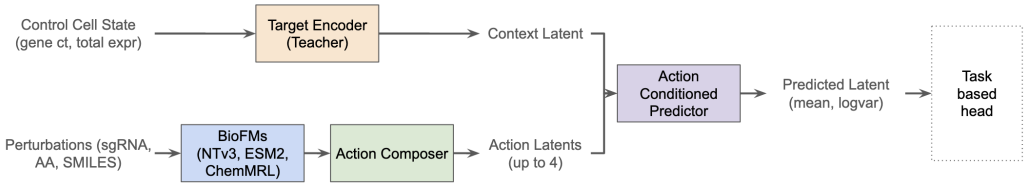

Fig. A1: *An overview of the BioJEPA-AC v1.0 inference flow, showing how we convert a control cell state and up to four perturbations into mean and log-variance latent representations of the predicted cell state.*

Our v1.0 architecture predicts a latent representation of cell state given its baseline cell state and up to four predefined or novel perturbations across different modalities. We represent cell states across 10,000 genes using normalized log-expression values that retain the extreme sparsity of the underlying single-cell RNA-seq data [7–10]. Our cell state encoder uses 256-dimensional embeddings, 6 layers, 4 attention heads, and a SwiGLU MLP ratio of 4.0, giving it 7,979,522 parameters. The student and teacher have the same architecture and parameter count; after training, we use only the teacher as the cell state encoder. The action composer contains 444,224 parameters. The action-conditioned predictor contains 2,565,760 parameters and uses a smaller 128-dimensional internal representation with 4 layers and 4 attention heads, projecting its inputs from and its outputs back to the encoder's 256-dimensional latent space. Together, the cell state encoder, action composer, and predictor used during inference contain 10,989,506 parameters. We identified these dimensions via Bayesian optimization with Optuna [29]. The architecture comprises the following modules:

- **Cell State Encoder**: A transformer-based module that maps cell states into a latent space based on relative gene expression and total cell expression. We train it using a context encoder (student) and an exponential-moving-average target encoder (teacher), then use the teacher to encode cell states throughout the rest of the model.
- **Action Composer**: A dual-pathway encoder with additive fusion and FiLM-based mode conditioning that creates a unified latent space for the embedding representations of different perturbation types.
- **Action-Conditioned Predictor**: A transformer-based module that uses the action latents to adjust the cell state representation in the latent space, generating the latent representation of the perturbed cell state.

A.2 Cell State Encoder

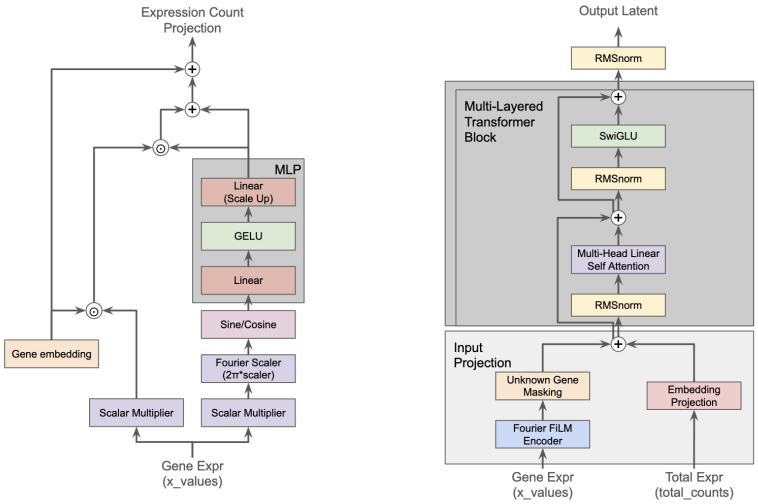

Fig. A2: *The data we use to create our cell state representation in the latent space.*

The cell state encoder is the foundation of our joint-embedding architecture [1]. We train it using a context encoder (student) and an exponential-moving-average target encoder (teacher) [27]. After encoder training, we use the frozen teacher to encode cell states throughout the rest of the model. The encoder is transformer-based [23] with gated linear self-attention [24], SwiGLU [3], and RMSNorm [25]. We build each gene token by adding a learned gene identity embedding to an expression-dependent representation of that gene. We scale the gene embedding by its normalized expression value, then use Gaussian Fourier features [26] of that same value (scale=5.699) to generate FiLM conditioning. The linear expression scaler is initialized to 0.6769. Separately,

we project the normalized total expression through a linear layer and add it to every gene token. This gives the model both the relative expression of each gene and the overall expression level of the cell, helping it distinguish cells with similar expression profiles but different total counts. Our cell representation lives in the $[\text{token}, \text{embedding}]$ space, where tokens correspond to genes. We designed the architecture so we can expand to non-gene-based tokens. Review our explainer notebook for more detail.

A.3 ACTION COMPOSER

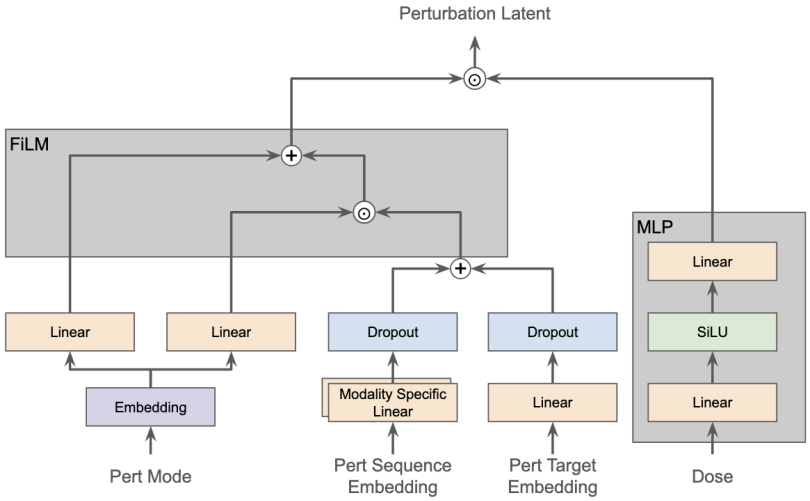

Fig. A3: *An overview of how we merge different perturbation types and data sources into the same perturbation representation.*

The action composer is a dual-pathway linear encoder that projects perturbation embeddings into a shared latent space via separate linear projections, fuses them, and applies FiLM [4] conditioning from a learned mode embedding to encode the perturbation with mechanism awareness. When dosage is available, we apply a log1p transformation before passing it to the composer. The composer uses a small dose network to generate a multiplicative scale for the action representation after mode conditioning. We use -1 to identify perturbations without dosage information. Our perturbation representation ends in the $[\text{n\_perts}, 128]$ space, with a mode embedding dimension of 64. We identified these dimensions via Bayesian optimization with Optuna [29].

Perturbations can be targeted genetic (e.g., CRISPR) perturbations or therapeutics including nucleic acids, proteins, and small molecules. We require that a perturbation has a sequence and/or target (at least one) and a mode (crispri, crispra, overexpression, knockout, inhibitor, agonist, degrader, binder, unknown). To improve flexibility and generalization, we pass the raw sequence and target representations through biological foundation models [5, 6] during data preparation rather than feeding them directly to the composer. We rely on the action composer to learn embeddings that represent functionally similar but sequence-different perturbations, enabling expansion to unseen perturbations. Review our explainer notebook for more detail.

A.4 ACTION-CONDITIONED PREDICTOR

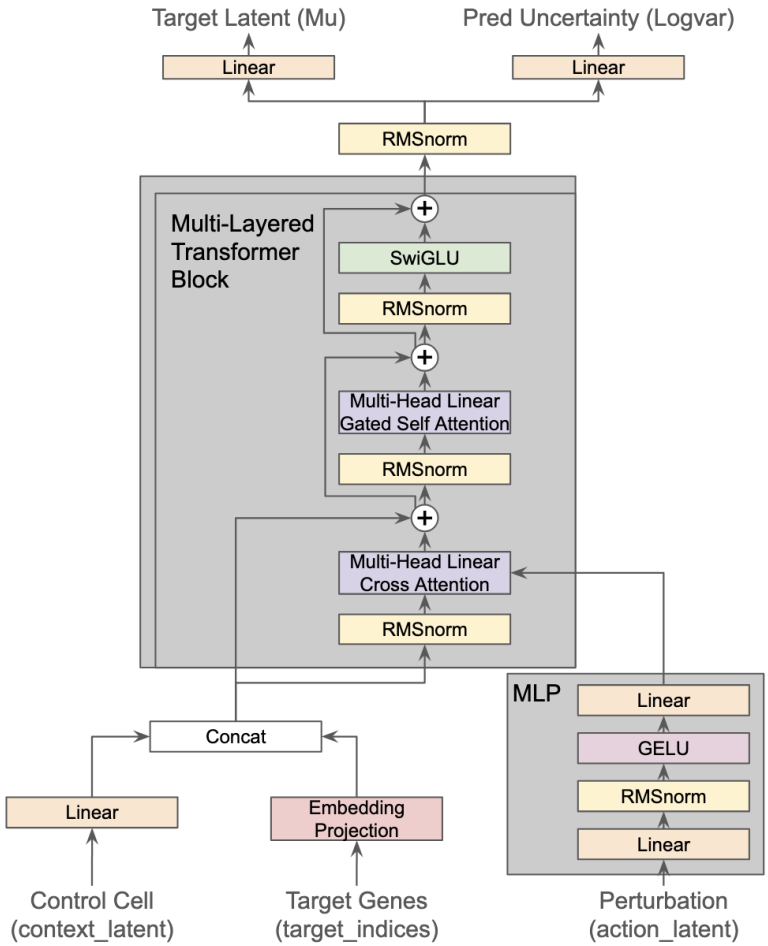

Fig. A4: *An overview of how our action-conditioned predictor uses the control cell state, all 10,000 target gene queries, and up to four perturbations to predict the perturbed cell state in our shared latent space.*

The action-conditioned predictor is the foundation of the "AC" portion of our BioJEPA-AC model. The predictor is transformer-based with similar linear attention, SwiGLU, and RMSNorm. It differs from the cell state encoder by employing two different attention layers: cross-attention to shift the cell state based on the perturbation, and self-attention to allow the cell state to learn from itself. The predictor creates a learned target token for each of the 10,000 genes and predicts a complete perturbed cell representation. During training, masking changes how much of the control cell is available as context, but the predictor continues to output a latent representation for every gene.

We first project the 10,000 control cell tokens from the encoder's 256-dimensional latent space into the predictor's 128-dimensional internal space. We concatenate them with the 10,000 learned gene-query tokens, creating a 20,000-token representation. The predictor processes this representation through four transformer blocks using action cross-attention, self-attention, and SwiGLU. We then retain the 10,000 gene-query tokens and project them back into the encoder's 256-dimensional latent space through separate mean and log-variance heads. The log-variance provides a latent uncertainty signal for each dimension of each gene token. The expression decoder uses the predicted mean, while we reserve the latent log-variance for uncertainty analysis. The predictor operates in the same shared latent space as the cell state encoder, learning where to move the representation based on the perturbations.

Review our explainer notebook for more detail.

REFERENCES

[1] LeCun, Y. (2022). A Path Towards Autonomous Machine Intelligence. *Position paper*. openreview.net

[2] Dixit, A. et al. (2016). Perturb-Seq: Dissecting Molecular Circuits with Scalable Single-Cell RNA Profiling of Pooled Genetic Screens. *Cell*, 167(7), 1853-1866.e17. doi:10.1016/j.cell.2016.11.038

[3] Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv:2002.05202

[4] Perez, E. et al. (2018). FiLM: Visual Reasoning with a General Conditioning Layer. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1). doi:10.1609/aaai.v32i1.11671

[5] Boshar, S. et al. (2025). A foundational model for joint sequence-function multi-species modeling at scale for long-range genomic prediction. *bioRxiv*. doi:10.64898/2025.12.22.695963

[6] Lin, Z. et al. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123-1130. doi:10.1126/science.ade2574

[7] Adamson, B. et al. (2016). A Multiplexed Single-Cell CRISPR Screening Platform Enables Systematic Dissection of the Unfolded Protein Response. *Cell*, 167(7), 1867-1882.e21. doi:10.1016/j.cell.2016.11.048

[8] Replogle, J.M. et al. (2022). Mapping information-rich genotype-phenotype landscapes with genome-scale Perturb-seq. *Cell*, 185(14), 2559-2575.e28. doi:10.1016/j.cell.2022.05.013

[9] Norman, T.M. et al. (2019). Exploring genetic interaction manifolds constructed from rich single-cell phenotypes. *Science*, 365(6455), 786-793. doi:10.1126/science.aax4438

[10] Srivatsan, S.R. et al. (2020). Massively multiplex chemical transcriptomics at single-cell resolution. *Science*, 367(6473), 45-51. doi:10.1126/science.aax6234

[11] Roohani, Y. et al. (2024). Predicting transcriptional outcomes of novel multigene perturbations with GEARS. *Nature Biotechnology*, 42, 927-935. doi:10.1038/s41587-023-01905-6. github.com/snap-stanford/GEARS

[12] Bardes, A. et al. (2022). VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning. *ICLR*. arXiv:2105.04906

[13] van den Oord, A. et al. (2018). Representation Learning with Contrastive Predictive Coding. arXiv:1807.03748

[14] Seitzer, M. et al. (2022). On the Pitfalls of Heteroscedastic Uncertainty Estimation with Probabilistic Neural Networks. *ICLR*. arXiv:2203.09168

[15] Cortes, E. (2025). ChemMRL: SMILES Matryoshka Representation Learning Embedding Transformer. *Model release*. huggingface.co/Derify/ChemMRL

[16] Bai, D. et al. (2024). AttentionPert: accurately modeling multiplexed genetic perturbations with multi-scale effects. *Bioinformatics*, 40(Supplement 1), i453-i461. doi:10.1093/bioinformatics/btae244

[17] Assran, M. et al. (2025). V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. arXiv:2506.09985

[18] National Center for Biotechnology Information. (n.d.). PubChem. pubchem.ncbi.nlm.nih.gov

[19] Wu Lab, Scripps Research. (n.d.). MyGene.info. mygene.info

[20] The UniProt Consortium. (n.d.). UniProt. uniprot.org

[21] National Center for Biotechnology Information. (n.d.). Entrez Search and Retrieval System. ncbi.nlm.nih.gov

[22] EMBL-EBI. (n.d.). Ensembl Genome Browser. ensembl.org

[23] Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS*. arXiv:1706.03762

[24] Katharopoulos, A. et al. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. *ICML*. arXiv:2006.16236

[25] Zhang, B. and Sennrich, R. (2019). Root Mean Square Layer Normalization. *NeurIPS*. arXiv:1910.07467

[26] Tancik, M. et al. (2020). Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains. *NeurIPS*. arXiv:2006.10739

[27] Assran, M. et al. (2023). Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture. *ICCV*. arXiv:2301.08243

[28] Smith, L.N. and Topin, N. (2018). Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. arXiv:1708.07120

[29] Akiba, T. et al. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *KDD*. arXiv:1907.10902